# Table of Contents

MAE 6760 Model Based Estimation ..... 1
User Inputs ..... 1
Define System Model & Simulate ..... 1
Run the EKF ..... 4
Generate filter plots ..... 5
--------- start-up items ..... 8

# MAE 6760 Model Based Estimation

Cornell University M Campbell

Homework #4 Problem #1: Extended Kalman Filter (EKF) planar pose of a car (four states) with added IMU biases nonlinear dynamics, IMU and 2D position (GPS-like) measurements Plotting uses plot_openloop.m and plot_estimator.m

```
%close all;
clear all;
MAE6760startup; %can adjust font size, figure size at the bottom of this
script
global MCcolors; %define colors as global with access
rng(100);
```

# User Inputs

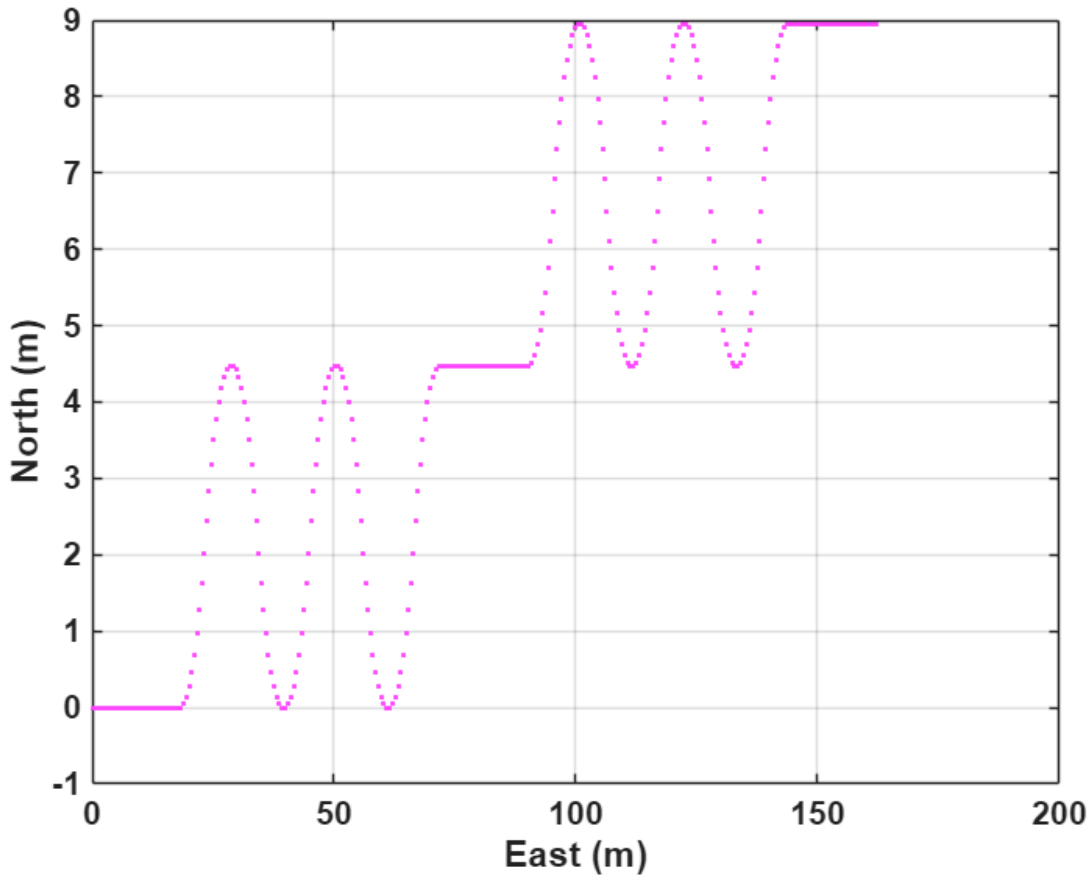
Two cases to consider: select either baseline or swervy

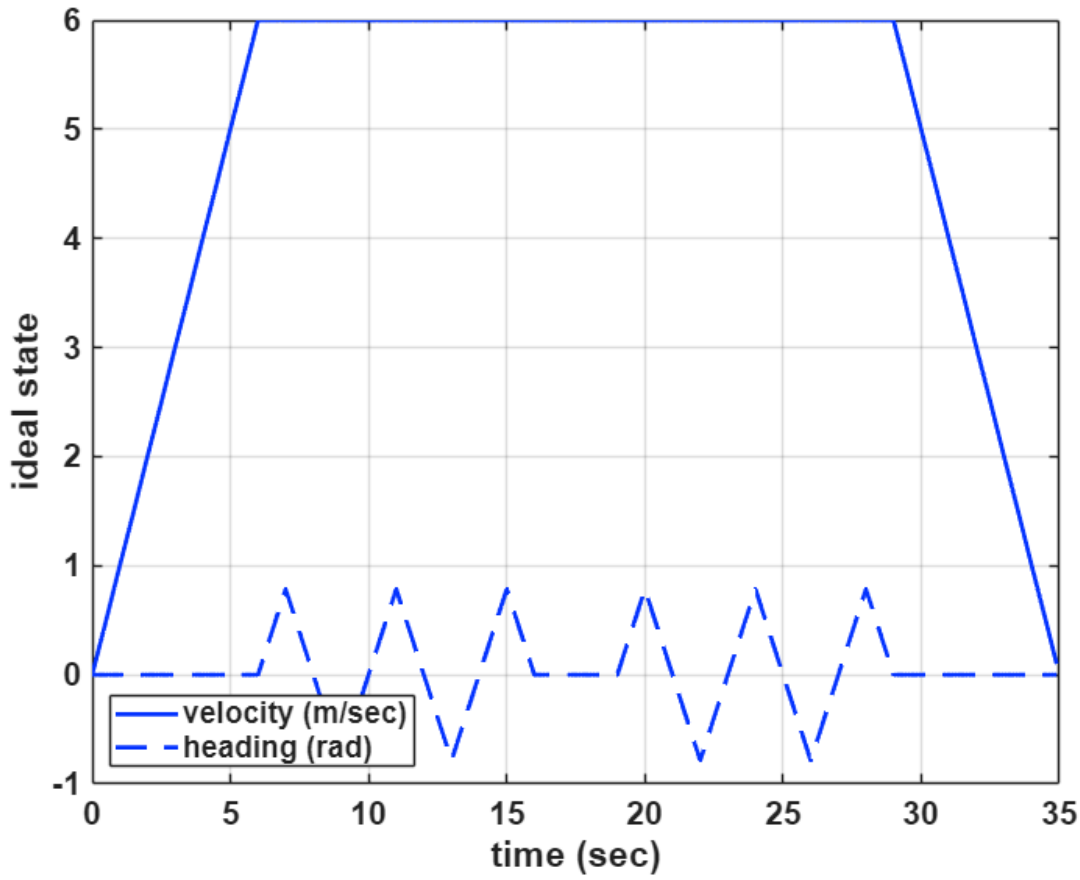
```
scenario_type='baseline';
scenario_type='swervy';
```

# Define System Model & Simulate

```
%car inputs: acceleration and heading rate
[Uacc,Uomega]=get_controlinputs(scenario_type);
%
%state vector: [x,y, velocity, heading]
%simulate no noise system
dt=0.1;
nt=length(Uacc);
t=[0:dt:dt*(nt-1)];
nx=4;
x_true=zeros(nx,nt);
for k=1:(nt-1),
    Vk=x_true(3,k);
    Tk=x_true(4,k);
    x_true(:,k+1) = x_true(:,k) +...
        dt*[Vk*cos(Tk);Vk*sin(Tk);Uacc(k);Uomega(k)];
end
%
```

```
%birds-eye-view of the 2D car position
figure;
pt=plot(x_true(1,:),x_true(2,:),'.','color',MCcolors.mag,'LineWidth',3);
%true x,y position
grid
ylabel('North (m)');xlabel('East (m)');

%true car state velocity and heading over time
figure;
pp=plot(t,x_true(3,:),'-',t,x_true(4,:),'--','Color',MCcolors.blue);%,'LineWi
dth',3);
ylabel('ideal state');xlabel('time (sec)');
legend('velocity (m/sec)','heading (rad)','Location','Southwest');
grid

%Create process noise, and accel/rate gyro measurements
bias_acc=0.1; %accel bias
bias_rg=-0.025; %rg bias
x_true=[x_true;bias_acc*ones(1,nt);bias_rg*ones(1,nt)]; %update true states
nx=nx;
nw=2;
Q=diag([0.1^2 0.04^2]);
w=sqrtm(Q)*randn(nw,nt);
Zacc=Uacc+bias_acc+w(1,:);
Zrg=Uomega+bias_rg+w(2,:);

%Create 2D GPS measurements
nz=2;
R=eye(nz)*1^2;
v=sqrtm(R)*randn(nz,nt);
z=[x_true(1:2,:)] + v;
H=[eye(2) zeros(2,2)]; %Note: output matrix is linear
```


9
8
7
6
5
4
3
2
1
0
-1
North (m)
0
50
100
150
200
East (m)




# Run the EKF

```
x0 = [0;0;0;0];
P0_vals = [2^2 2^2 1^2 0.1^2];
P0 = diag(P0_vals);
xhatu = zeros(nx,nt); xhatu(:,1)=x0;
Pu = zeros(nx,nx,nt); Pu(:,:,1)=P0;
xhatp=xhatu;
Pp=Pu;

for k=1:(nt-1)
    % predict state
    xhatp(:,k+1) = predict_state_carposebias(xhatu(:,k), [Zacc(k); Zrg(k)],
dt);

    % predict covariance
    [F,G] = getFG_carposebias(xhatu(:,k), dt);
    Pp(:,:,k+1) = F * Pu(:,:,k) * F' + G * Q * G';

    % Kalman Gain
    K = Pp(:,:,k+1)*H'*inv(H*Pp(:,:,k+1)*H' + R);
```

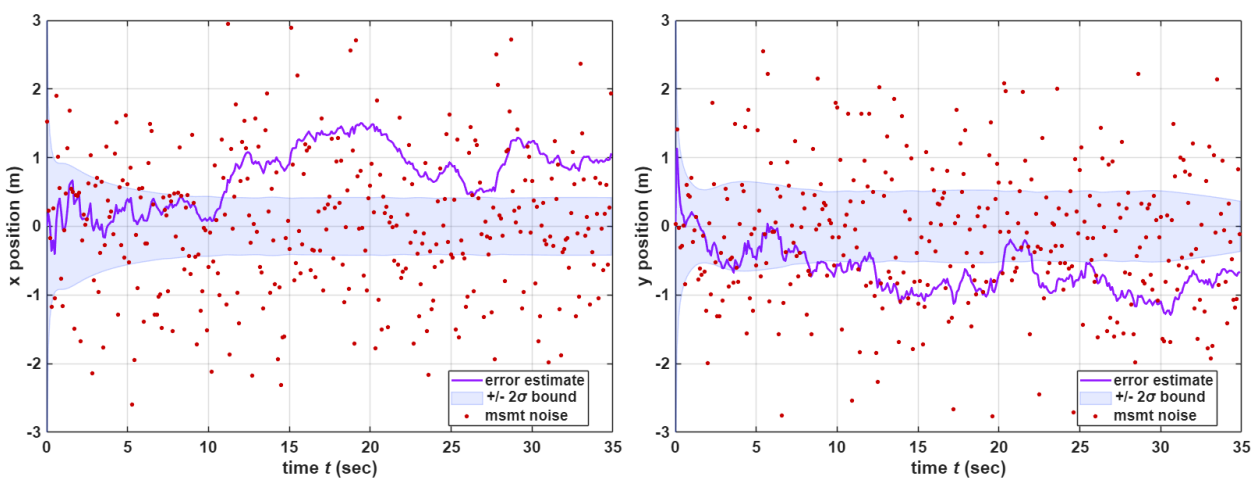
```
    % update state with measurement
    xhatu(:,k+1) = xhatp(:,k+1) + K * (z(:,k+1) - H * xhatp(:,k+1));
    Pu(:,:,k+1) = (eye(nx) - K * H) * Pp(:,:,k+1);
end
```

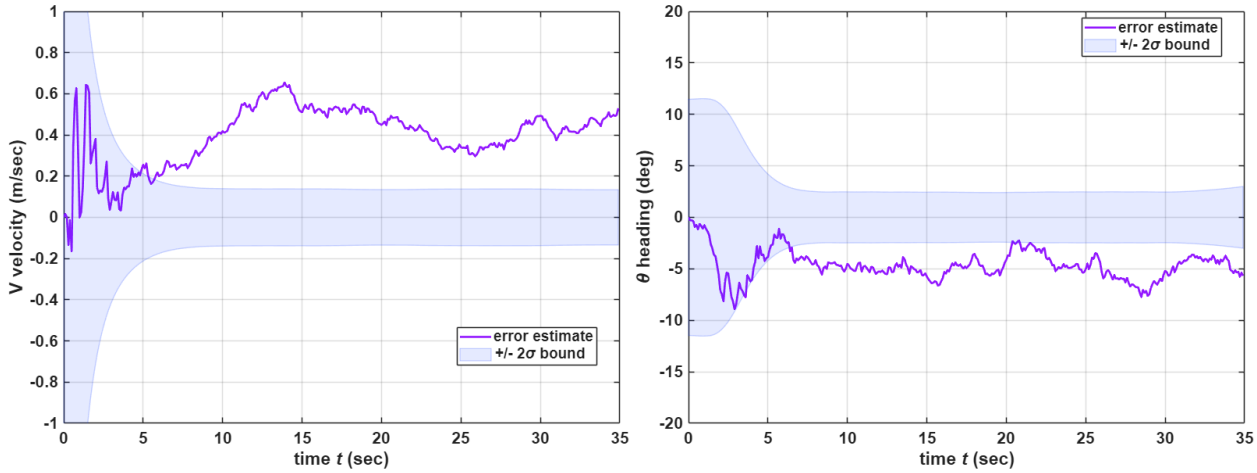
# Generate filter plots

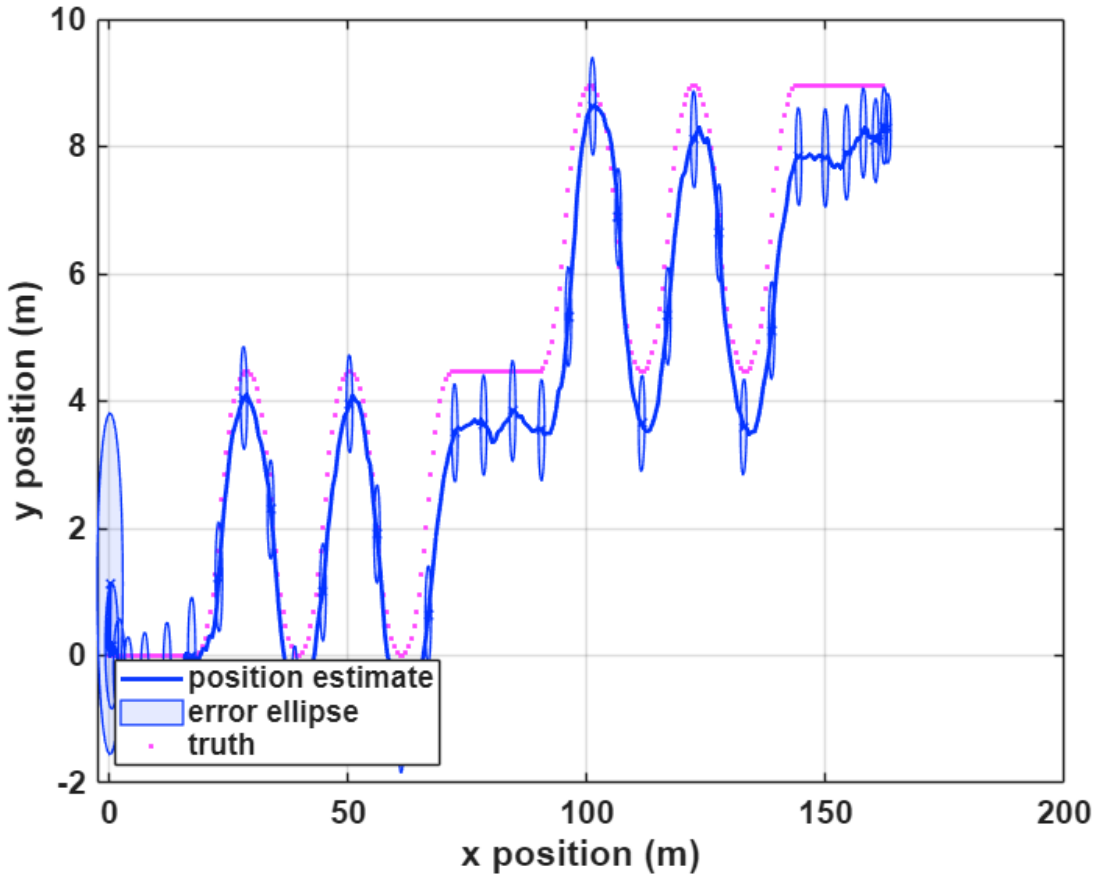
```
%x,y position errors and 2-sigma plots over time
figure('Position',[100 100 1600 600]);
tiledlayout(1,2,'TileSpacing','compact','Padding','tight');
nexttile;
iix=1;
plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error',z(1,:))
ylabel('x position (m)');
axis([0 35 -3 3]);
iix=2;
nexttile;
plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error',z(2,:))
ylabel('y position (m)');
axis([0 35 -3 3]);

%V,th errors and 2-sigma plots over time
figure('Position',[100 100 1600 600]);
tiledlayout(1,2,'TileSpacing','compact','Padding','tight');
nexttile;
iix=3;
plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error')
ylabel('V velocity (m/sec)');
axis([0 35 -1 1]);
iix=4;
nexttile;
thC=180/pi;
plot_estimator(t,xhatu(iix,:)*thC,Pu(iix,iix,:)*thC^2,x_true(iix,:)*thC,'erro
r')
ylabel('\theta heading (deg)');
axis([0 35 -20 20]);

% %bias errors and 2-sigma plots over time
% figure('Position',[100 100 1600 600]);
% tiledlayout(1,2,'TileSpacing','compact','Padding','tight');
% nexttile;
% iix=5;
% plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error')
% ylabel('accel bias (m/sec^2)');
% iix=6;
% nexttile;
% plot_estimator(t,xhatu(iix,:),Pu(iix,iix,:),x_true(iix,:),'error')
% ylabel('rate gyro bias (rad/sec)');

%birds-eye-view of the 2D car position and 2D 2-sigma uncertainty ellipsoids
figure;
pt=plot(x_true(1,:),x_true(2,:),'.','color',MCcolors.mag,'LineWidth',2);
%true x,y position
```

```
hold on;
ph=plot(xhatu(1,:),xhatu(2,:),'-','color',MCcolors.blue); %state estimate
%plot 2-sigma uncertainty ellipsoids at a few positions along trajectory
iell=[2 10:10:350];
for i=1:length(iell),
    ii=iell(i);
    [Xe,Ye] = calculate_ellipse(xhatu([1 2],ii),Pu([1 2],[1 2],ii),3);

pu(i)=plot(xhatu([1],ii),xhatu([2],ii),'x','color',MCcolors.blue,'LineWidth',
1);
    ne=length(Xe);
    x1=[Xe((ne/2+1):ne);Xe(1:ne/2)];
    y1=[Ye((ne/2+1):ne);Ye(1:ne/2)];

pp(i)=patch(x1,y1,'b','facecolor',MCcolors.blue,'edgecolor',MCcolors.blue,'li
newidth',1,'FaceAlpha',.1,'EdgeAlpha',1);

end
xlabel('x position (m)');ylabel('y position (m)');
hold off;
grid
legend([ph pp(1) pt],'position estimate','error
ellipse','truth','Location','Southwest');

function Xkp1=predict_state_carposebias(Xk,U,dt);
%
%

Vk = Xk(3);
Tk = Xk(4);

acc = U(1);
omegadot = U(2);

    Xkp1 = Xk + dt*[Vk*cos(Tk);
        Vk*sin(Tk);
        acc;
        omegadot];

%
end

function [F,G]=getFG_carposebias(X,dt);
%
%
Vk = X(3);
Tk = X(4);

    F = [1, 0, dt*cos(Tk), -dt*Vk*sin(Tk);
        0, 1, dt*sin(Tk), dt*Vk*cos(Tk);
        0, 0, 1, 0;
        0, 0, 0, 1];

    G = [0, 0;
```

```
        0, 0;
        dt, 0;
        0, dt];

%
end

function [Uacc,Uomega]=get_controlinputs(scenario_type);
%
%scenario_type='baseline';
%scenario_type='swervy';
%
%   generates clean acceleration and rotational rate control inputs
%
%
%define inputs: acceleration and heading rate
if strcmp(scenario_type,'baseline'),
    Uacc=[ones(1,60) zeros(1,100) -ones(1,60) zeros(1,20) ones(1,60)
zeros(1,50)];
    Uomega=[zeros(1,240) -ones(1,40)*pi/2/4 zeros(1,70)];
elseif strcmp(scenario_type,'swervy'),
    Uacc=[ones(1,60) zeros(1,230) -ones(1,60)];
    Uomega=[zeros(1,60) ones(1,10)*pi/2/2 -ones(1,20)*pi/2/2
ones(1,20)*pi/2/2 ...
        -ones(1,20)*pi/2/2 ones(1,20)*pi/2/2 -ones(1,10)*pi/2/2
zeros(1,30) ...
        ones(1,10)*pi/2/2 -ones(1,20)*pi/2/2 ones(1,20)*pi/2/2 ...
        -ones(1,20)*pi/2/2 ones(1,20)*pi/2/2 -ones(1,10)*pi/2/2 ...
        zeros(1,60)];
else,
    return;
end
%
end
```






# --------- start-up items

```
function MAE6760startup(font_size);
%
% define colors for plotting
global MCcolors; %define colors as global with access
MCcolors.red=[200,0,0]/255;
MCcolors.blue=[4,51,255]/255;
```

```
MCcolors.purple=[147,23,255]/255;
MCcolors.green=[0,160,0]/255;
MCcolors.orange=[253,128,8]/255;
MCcolors.mag=[255,64,255]/255;
MCcolors.cyan=[0,230,255]/255;
%
% define standard figure positioning and size
set(groot,'DefaultFigureUnits','pixels');
set(groot,'DefaultFigurePosition',[100 100 800 600]);
set(groot,'DefaultFigureWindowStyle','normal');  % Important
set(groot,'DefaultAxesFontSize',16);
set(groot,'DefaultAxesFontWeight','bold');
set(groot,'DefaultLineLineWidth',2);
%
end
```

*Published with MATLAB® R2025b*